

From SI to System Agency

Fabric Storyboard Copilot — a Microsoft Fabric × PowerPoint case study
built with Squad + Copilot CLI

May 2026

Contents

1	Context	2
2	End-to-End Architecture	2
3	1. The Project — Fabric Storyboard Copilot	3
3.1	1.1 What it does	3
3.2	1.2 Tech stack	5
3.3	1.3 Auth flow (OBO)	6
3.4	1.4 One-click deployment	6
4	2. How It Was Built — The Squad	9
4.1	2.1 Eight specialised agents	9
4.2	2.2 Timeline — 13.3 hours, end to end	10
4.3	2.3 The integration bugs that ate the day	11
4.4	2.4 AI code review	11
5	3. The Tooling Ecosystem	11
5.1	3.1 Copilot CLI as the conductor	11
5.2	3.2 Spec Kit, Agent Forge, Agent Store	12
5.3	3.3 Fleet vs Squad — three patterns, one substrate	13
5.4	3.4 Spec-Driven Development — making the spec the source of truth	13
5.4.1	Three spec layers	13
5.4.2	Base template (every spec)	14
5.4.3	Contracts are first-class citizens	14
5.4.4	The SDD workflow	14
5.4.5	Suggested repository layout	14
5.4.6	Governance — avoiding drift	15
5.4.7	Agents + SDD = high-fidelity context	15
5.4.8	Where to start — don't boil the ocean	15
5.5	3.5 Agents vs Skills	16
5.6	3.6 Coding with agents — best practices	16
5.7	3.7 Testing strategy	17
6	4. The Cost Model	17
6.1	4.1 AI Credits	17
6.2	4.2 Cost comparison	18
7	5. From SI to System Agency	18
7.1	5.1 The shift	18
7.2	5.2 Why now	18
7.3	5.3 Old world vs new world	19
7.4	5.4 Three phases	19
7.5	5.5 Encoding IP	19
7.6	5.6 Emerging roles	19
7.7	5.7 Getting started	19
7.8	5.8 Lessons learned	19

8 6. Conclusion

20

9 Resources

20

1 Context

Fabric Storyboard Copilot is a PowerPoint Office Add-in that turns Microsoft Fabric / Power BI content into board-ready slides in two clicks. It browses workspaces, exports report pages as PNG into the active slide, and generates an executive narrative through Azure OpenAI GPT-4o Vision — all from inside PowerPoint Desktop or Web, with single sign-on against the user's Entra identity.

The product itself is interesting. **The way it was built is the real story.** A solo developer assembled a Squad of eight Copilot agents, supervised them with the GitHub Copilot CLI, and shipped a production-grade add-in — Bicep infrastructure, Entra app registration, GPT-4o Vision integration, end-to-end tests, AI code review — in **13.3 hours**. The same scope estimated by traditional methods: **43 man-days**. A **26× compression**.

This document captures both halves: the project (what was built, how it works) and the methodology (Spec Kit → Agent Forge → Squad → Copilot CLI → production). It closes on the broader thesis the project illustrates: **the System Integrator model is collapsing into a System Agency model** — selling outcomes instead of man-days, with margin structure inverting from 30 % to 90 %+.

Source repository: <https://github.com/fredgis/OfficeAddin>.

2 End-to-End Architecture

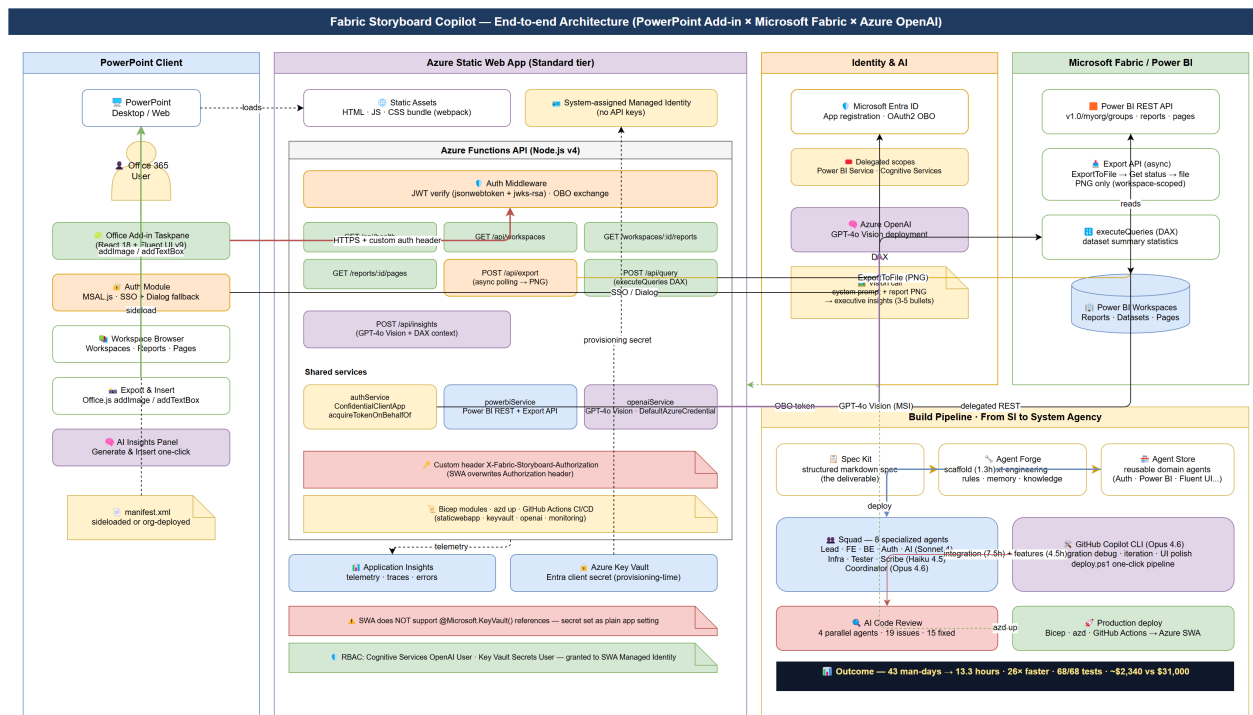


Figure 1: Fabric Storyboard Copilot Architecture

The diagram above traces the complete request path from PowerPoint client to Microsoft Fabric, plus the build pipeline that produced the application. Five lanes:

1. **PowerPoint Client** — Office Add-in taskpane (React 18 + Fluent UI v9), MSAL.js authentication module with SSO and dialog fallback, Office.js APIs for slide manipulation. Distributed as a sideloaded `manifest.xml` or organisation-deployed catalog.
2. **Azure Static Web App (Standard tier)** — hosts both the static bundle and the integrated Azure Functions API. System-assigned managed identity gives the Functions runtime keyless access to Azure OpenAI and Key Vault.
3. **Identity & AI** — Microsoft Entra ID issues delegated tokens for Power BI and Cognitive Services scopes. Azure OpenAI runs a GPT-4o Vision deployment that ingests report PNG + DAX summary statistics and returns a 3-to-5-bullet executive narrative.
4. **Microsoft Fabric / Power BI** — the data plane: Power BI REST API for navigation, Export-to-File API (async PNG export), `executeQueries` for DAX summaries.
5. **Build Pipeline** — Spec Kit captures the structured spec, Agent Forge engineers the context (rules, memory, knowledge), Agent Store provides reusable domain agents, the Squad of eight specialised agents scaffolds and codes, the Copilot CLI handles integration debugging, an AI code-review pass surfaces 19 issues (15 fixed), and `azd up` deploys via Bicep + GitHub Actions.

3 1. The Project — Fabric Storyboard Copilot

3.1 1.1 What it does

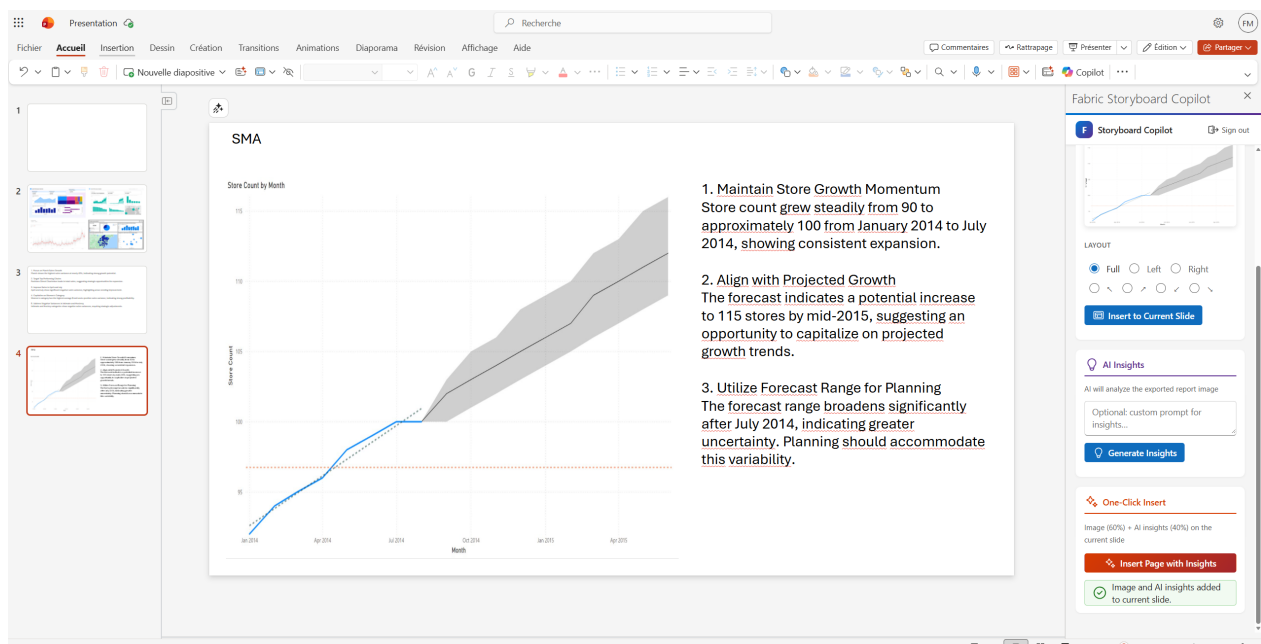


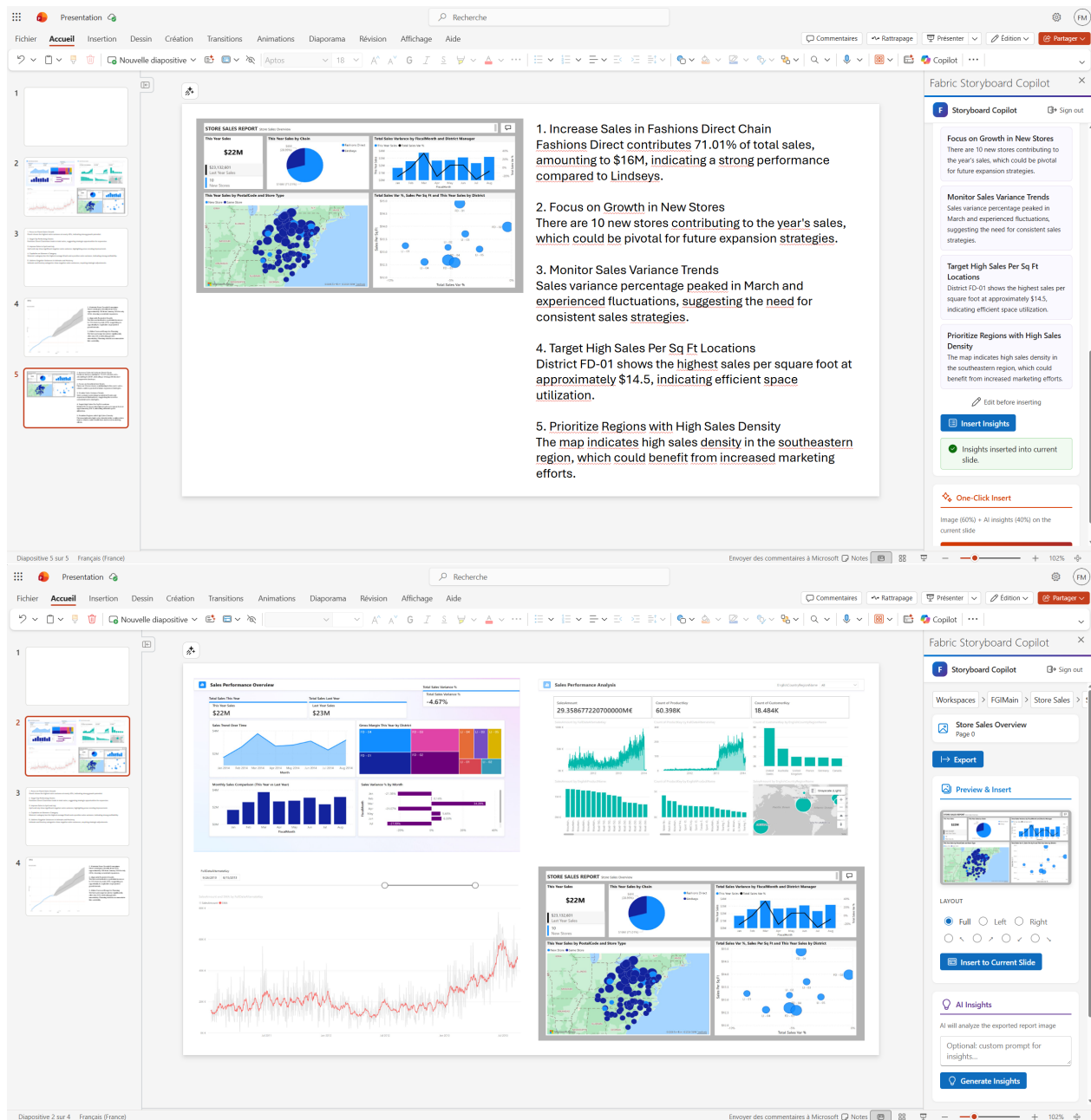
Figure 2: Taskpane experience

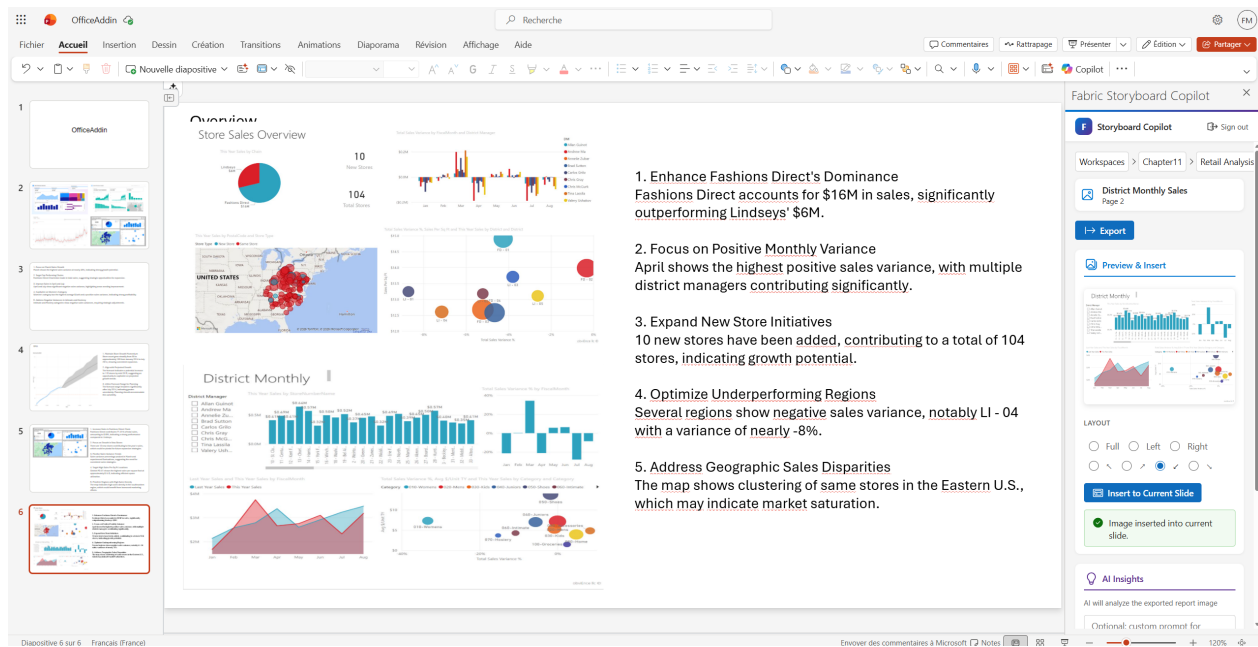
Three core experiences inside the taskpane:

- **Browse & Insert** — list Power BI workspaces and reports the user has access to, expand to page level, click *Insert as image*. The selected page is exported as a PNG via the Power BI

Export-to-File API and dropped into the current slide via `Office.js addImageFromBase64`.

- **AI Insights** — the same page is sent to GPT-4o Vision along with summary statistics computed from `executeQueries` (DAX). The model returns 3-5 executive bullets which are inserted as a Fluent-styled text box next to the chart.
- **Single sign-on** — MSAL acquires an Entra token using Office SSO when available; falls back to a popup dialog for browsers/tenants that disable nested auth.





3.2 1.2 Tech stack

Layer	Technology	Why
Frontend	React 18 · TypeScript · Fluent UI v9 · MSAL.js	Native Microsoft 365 look; SSO support
Backend	Azure Functions v4 (Node.js 20) · Azure SDK	Per-route handlers, OBO token exchange, no key rotation
AI	Azure OpenAI · GPT-4o Vision	Multimodal (image + text) one-shot insight generation
Identity	Microsoft Entra ID · OAuth 2.0 OBO	Delegated permissions, no client secret in the taskpane
Hosting	Azure Static Web App (Standard)	Static + API in one resource; managed identity built in
Secrets	Azure Key Vault + System-assigned MSI	RBAC, no API keys in code or settings
Observability	Application Insights	Distributed traces taskpane → API → AOAI
IaC	Bicep modules · azd · GitHub Actions	One-command provisioning + CI/CD

3.3 1.3 Auth flow (OBO)

1. Taskpane calls `OfficeRuntime.auth.getAccessToken({ allowSignInPrompt: true })` (SSO) or falls back to `OfficeRuntime.auth.openDialog` (popup).
2. The Entra access token is sent to the Functions API in a custom header `X-Fabric-Storyboard-Authorization` — Static Web App overwrites the standard `Authorization` header, so we use a custom one.
3. The `authMiddleware` validates the JWT using `jsonwebtoken + jwks-rsa`.
4. `authService.acquireTokenOnBehalfOf(...)` exchanges that token for a Power BI or Azure OpenAI access token using the OBO flow.
5. Downstream calls (Power BI REST, Export API) use the OBO token. Azure OpenAI uses `DefaultAzureCredential` (managed identity), not OBO — Vision is treated as a service capability, not a per-user resource.

```
// api/middleware/auth.js - verified JWT, returned in req.user
const decoded = jwt.verify(token, getKey, {
  audience: process.env.ENTRA_API_AUDIENCE,
  issuer: `https://login.microsoftonline.com/${tenantId}/v2.0`,
  algorithms: ['RS256']
});
req.user = decoded;
req.userToken = token; // forwarded to OBO
```

3.4 1.4 One-click deployment

```
.\deploy.ps1 -EnvName fabric-storyboard-prod -Location westeurope
# → azd provision (Bicep)
# → Entra app registration + scope grant
# → manifest.xml regeneration with deployed URLs
# → azd deploy (SWA + Functions)
# → smoke test /api/health
```

```

Subscription:
Location: East US 2

You can view detailed progress in the Azure Portal:
https://portal.azure.com/#view/HubsExtension/DeploymentDetailsBlade/~/overview/id/%2Fsubscriptions/ra

(✓) Done: Resource group: rg-fabricaddina (7.596s)
(✓) Done: Key Vault: (24.916s)
(✓) Done: Log Analytics workspace: log-appoi (28.104s)
(✓) Done: Application Insights: appi-s (8.275s)
(✓) Done: Static Web App: swa:

SUCCESS: Your application was provisioned in Azure in 2 minutes 1 second.
You can view the resources created under the resource group
https://portal.azure.com/#@/resource/subscriptions,
WARNING: your version of azd is out of date, you have 1.23.12 and the latest stable version is 1.23.13

To update to the latest version, run:
winget upgrade Microsoft.Azd

✓ Resource group: rg-fabricaddina
✓ Static Web App: swa
✓ Key Vault: kv
✓ URL: https://

▶ Deploying application to Azure

Welcome to Azure Static Web Apps CLI (2.0.8)

Deploying front-end files from folder:
C:\Users\frgisber\repo-addinfabric\OfficeAddin\dist

Deploying API from folder:
C:\Users\frgisber\repo-addinfabric\OfficeAddin\api

Consider providing api-language and version using --api-language and --api-version flags,
otherwise default values apiLanguage: node and apiVersion: 16 will apply

Deploying to environment: production

Deploying project to Azure Static Web Apps...
- Preparing deployment. Please wait...
✓ Deployed to https://.azurestaticapps.net

▶ Generating production manifest
✓ dist/manifest-prod.xml

▶ Adding production redirect URI to Entra app
✓ https://.azurestaticapps.net/dialog.html

▶ Generating local development settings
✓ api/local.settings.json updated
✓ .env updated

▶ Setting up RBAC for local dev (Cognitive Services OpenAI User)
⚠ Could not find OpenAI resource - assign role manually

[✓] Deployment Complete!

Application URL
Environment
Resource Group
Entra Client ID
Entra Client Secret
Tenant ID
Key Vault
Production Manifest

```



```

Verifying deployed frontend x PowerShell x + v
| BLACKGRID [15:32:46] [main] > .\deploy.ps1

Fabric Storyboard Copilot - One-Click Deploy

▶ Checking prerequisites
  ✓ Node.js v24.14.0
  ✓ npm v11.9.0
  ✓ Azure CLI v
  ✓ Azure Developer CLI (azd)

▶ Authenticating with Azure
  ✓ Signed in as
  ✓ Subscription:
  Authenticating azd (fresh login)...
  ✓ azd authenticated

▶ Gathering configuration
  Environment name (e.g., dev, staging, prod) [dev]:
  ✓ Environment: dev
  Azure region [eastus2]:
  ✓ Location: eastus2
  Deploy a NEW Azure OpenAI resource? (y/n) [n]:
  Existing Azure OpenAI endpoint URL: https://.openai.azure.com/openai/v1/
  ✓ OpenAI endpoint: https://.openai.azure.com/openai/v1/
  ✓ OpenAI deployment: gpt-4o

▶ Configuring Entra ID
  Create a NEW Entra ID app registration? (y/n) [y]:
  Registering app: Fabric Storyboard Copilot (dev)
  ✓ App registered:
  Setting Application ID URI...
  Exposing access_as_user scope...
  ✓ Scope: api:// /access_as_user
  Adding redirect URIs...
  Adding API permissions...
  Authorizing Office clients...
  ✓ Office clients authorized (SSO)
  Generating client secret...
  ✓ Client secret created (valid 1 year)
  Creating service principal...
  Granting admin consent...
  ✓ Entra ID setup complete

▶ Installing dependencies
  ✓ Frontend packages installed
  ✓ Backend packages installed

▶ Building application
  ✓ Frontend → dist/
  ✓ Backend → api/dist/

▶ Running tests
  ✓ Frontend tests passed
  ✓ Backend tests passed

▶ Provisioning Azure infrastructure (Bicep via azd)
WARNING: your version of azd is out of date, you have 1.23.12 and the latest stable version is 1.23.13

To update to the latest version, run:
winget upgrade Microsoft.Azd
WARNING: your version of azd is out of date, you have 1.23.12 and the latest stable version is 1.23.13

To update to the latest version, run:
winget upgrade Microsoft.Azd
WARNING: your version of azd is out of date, you have 1.23.12 and the latest stable version is 1.23.13

To update to the latest version, run:
winget upgrade Microsoft.Azd

```

4 2. How It Was Built — The Squad

4.1 2.1 Eight specialised agents

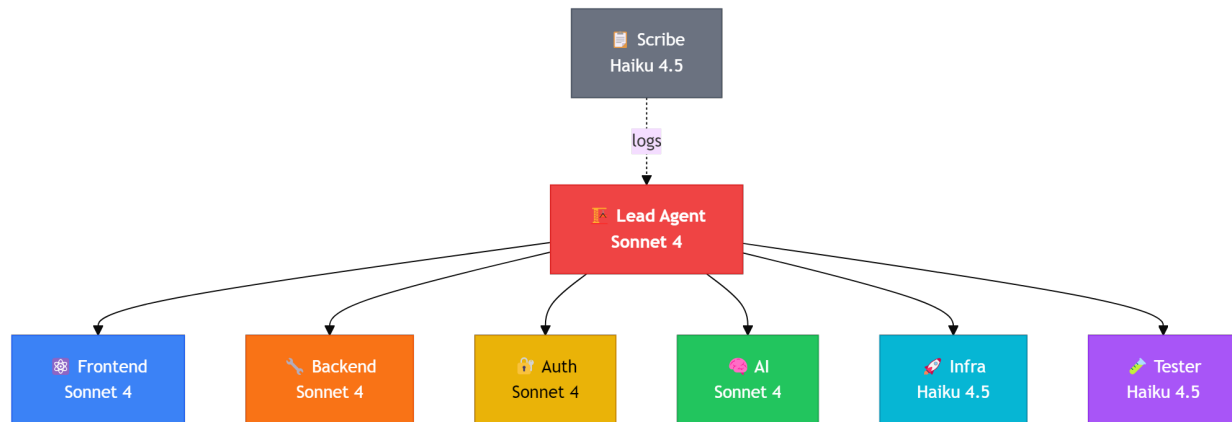


Figure 3: Squad of agents

The Squad is a roster of eight Copilot agents, each with a narrow role, tuned model, and curated context. Models are picked per role: heavy reasoning runs on Sonnet 4 / Opus 4.6, repetitive structural work runs on Haiku 4.5. Squad ships with five canonical roles (Lead, Frontend, Backend, Tester, Scribe); we extended the roster with **Auth, AI, Infra and Coordinator** agents pulled from the Agent Store to match the surface area of an Entra-protected Fabric add-in.

Agent	Model	Mission
Lead	Sonnet 4	Spec interpretation, task decomposition, coordination
Frontend	Sonnet 4	React 18 + Fluent UI v9 components, Office.js calls
Backend	Sonnet 4	Azure Functions endpoints, service layer, error handling
Auth	Sonnet 4	MSAL + Entra OBO, JWT validation, scope management
AI	Sonnet 4	Prompt engineering, GPT-4o Vision integration
Infra	Haiku 4.5	Bicep modules, RBAC, Key Vault, App Insights
Tester	Haiku 4.5	68 Jest + Playwright tests, fixtures, mocks
Scribe	Haiku 4.5	README, architecture docs, deployment guide
Coordinator	Opus 4.6	Cross-agent integration, conflict resolution

4.2 2.2 Timeline — 13.3 hours, end to end

Actual Effort Breakdown (13.3h total)

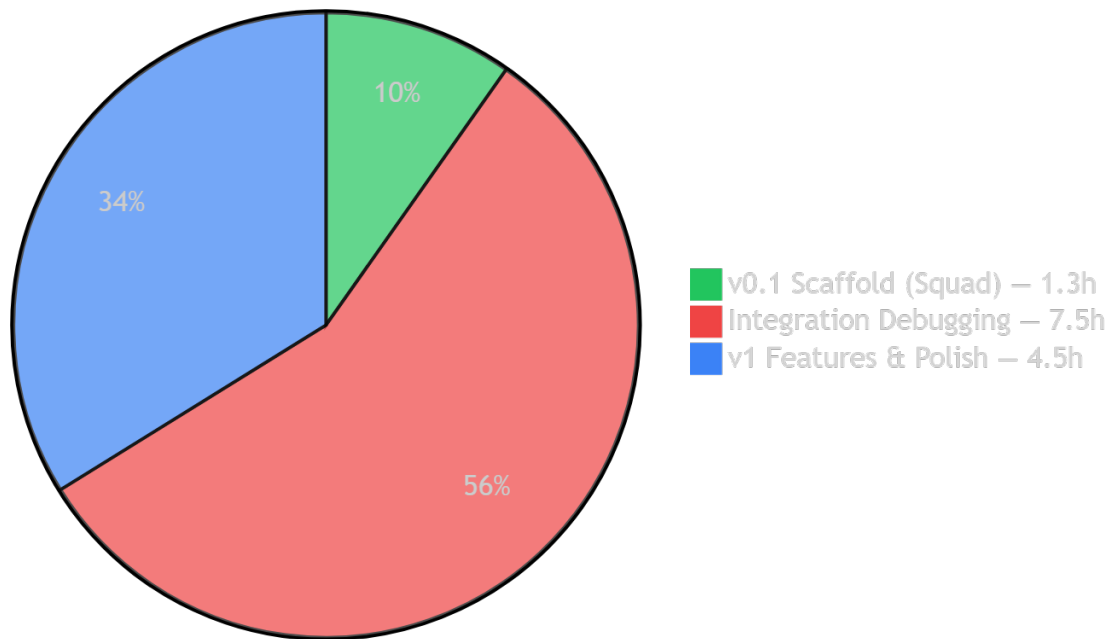


Figure 4: Effort distribution

Phase	Duration	What happened
Spec & Forge	1.0 h	Spec Kit document, agent context engineering
Squad scaffold	1.3 h	Eight agents in parallel — first compileable scaffold
Integration	7.5 h	Copilot CLI debugging the seams between agent outputs
V1 features	4.5 h	AI Insights, polish, telemetry, copy refinement
Code review	0.5 h	Four parallel review agents, 19 findings, 15 fixes
Deploy	0.5 h	azd up, manifest regeneration, smoke test
Total	13.3 h	vs 43 man-days estimated traditionally

The headline number masks where the work actually lives: **scaffold is fast, integration is slow**. Eight agents in parallel produce eight locally-correct fragments that don't always agree on contracts. Most of the 7.5 h integration phase was the Copilot CLI tracing why a JWT validated in one place was rejected in another, why a Bicep output didn't flow into an app setting, why the SWA dropped the `Authorization` header.

4.3 2.3 The integration bugs that ate the day

A representative sample of issues that the Squad produced and the CLI had to resolve:

1. **SWA strips Authorization.** Static Web App's auth proxy reserves the header for its own EasyAuth product. Solution: custom header `X-Fabric-Storyboard-Authorization`, validated in middleware.
2. **@Microsoft.KeyVault() not supported on SWA.** App settings can't reference Key Vault on Static Web App (works on App Service / Functions Premium). Solution: read the secret at provisioning time and inject as plain app setting; runtime calls Vault directly via MSI.
3. **Bicep circular dependency** between the SWA module (needed Entra app ID) and the Entra module (needed SWA URL for redirect URLs). Solution: two-pass deployment — SWA first with placeholder, Entra second, SWA app settings updated at the end.
4. **Office.js addImageFromBase64 size limit** (~ 1 MB). Solution: PNG resize to 1600 px wide before insertion.
5. **OBO refresh on long-running export.** Power BI Export-to-File can take 10-60 s; the user token can expire mid-poll. Solution: cache the OBO token and refresh once on **401**.

4.4 2.4 AI code review

Four parallel review agents (security, performance, style, architecture) ran on the final tree:

- **19 issues raised** — 6 security (e.g. JWT issuer not pinned in dev mode), 5 performance (missing memoisation in workspace tree), 4 style, 4 architecture.
- **15 fixed** within the same session. The 4 deferred were design decisions (e.g. “consider distributed cache” — out of scope for v1).

5 3. The Tooling Ecosystem

5.1 3.1 Copilot CLI as the conductor

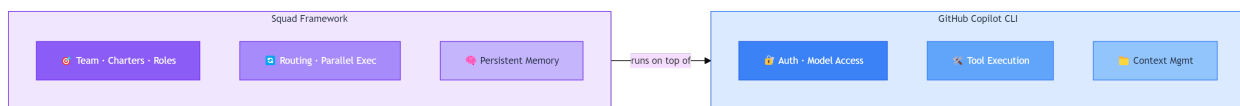


Figure 5: Copilot CLI × Squad

The Copilot CLI plays a different role than the Squad agents. The Squad produces code; the CLI **integrates, debugs, and iterates** with the developer in the loop. It is the conductor sitting between the human intent and the chorus of specialised agents.

5.2 3.2 Spec Kit, Agent Forge, Agent Store

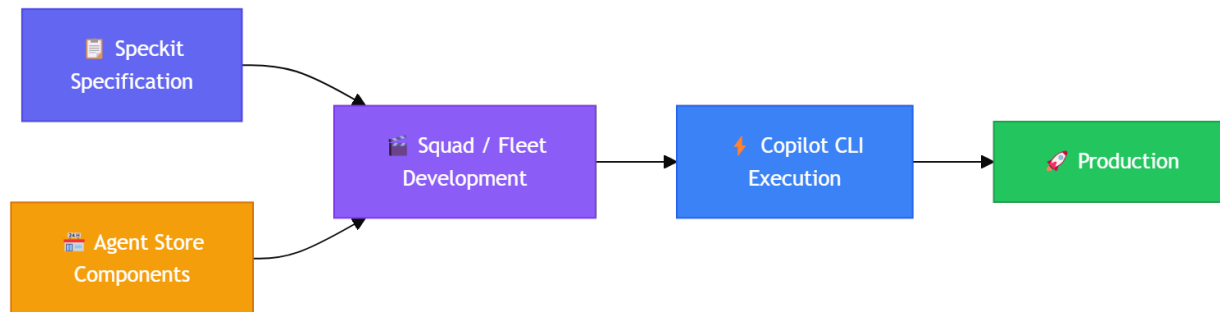


Figure 6: Toolchain

The toolchain is layered. Spec Kit captures intent; Agent Forge engineers the context the agents will see; the Agent Store keeps the reusable pieces alive across projects.

Spec Kit — structured markdown spec (problem, scope, contracts, acceptance criteria). The deliverable is the spec; the code is generated from it. Spec-first inverts the traditional flow.

Agent Forge — open-source context engineering toolkit from Microsoft ([microsoft/agent-forge](https://github.com/microsoft/agent-forge)). A multi-agent pipeline that *plans* → *generates* → *validates* → *installs* the Copilot customisation surface of a project. It emits a coherent set of artifacts:

- `.agent.md` — agent persona, model, tools, and responsibilities
- `.prompt.md` — slash-command (e.g. `/refactor`, `/spec`) routed to an agent
- `.instructions.md` — quality rules scoped by file glob (e.g. all `*.tsx` files)
- `SKILL.md` — reusable domain procedure called by agents
- `.vscode/mcp.json` — Model Context Protocol server bindings
- hooks — automation triggered by repo events (pre-commit, post-merge)

Two modes:

- **Greenfield** — Agent Forge takes a text description (“PowerPoint add-in with Power BI export and GPT-4o Vision insights”), decomposes it into domains, dispatches specialist sub-agents to each domain, and emits the full `.github/` + `.vscode/` configuration.
- **Brownfield** — Agent Forge scans an existing repo, infers domains from the structure (auth, services, infra, tests, docs), maps them to agent personas, and generates the same artifacts while preserving existing conventions.

A post-generation validator auto-fixes YAML front-matter, tool names, glob patterns, and content quality. The output is ready to commit.

Agent Store — reusable domain agents (Auth, Power BI, Fluent UI, Bicep). Pulled into the Squad for any project that touches the same surface area. Every project enriches the Store; every Store reuse compresses the next project. **The Store is the moat.**

5.3 3.3 Fleet vs Squad — three patterns, one substrate

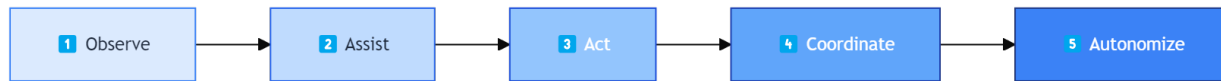


Figure 7: Scaling

Pattern	Scale	Coordination	Memory	Example
Solo	1 agent, 1 task	None	Single session	"Add a settings page"
Squad	5–10 agents, 1 project	Lead + Coordinator + CLI	Repo-resident (.squad/, .github/agents/)	This add-in: 8 agents, 13.3 h
Fleet	50–500 agents, 1 programme	Per-team Squad, central Agent Store, shared review pipeline	Org-wide Agent Store + per-repo memory	Enterprise migration across 30 services

Squads compose into Fleets **without changing the substrate**. The same agent definitions, prompts, memory and review pipeline scale from one developer to an enterprise programme. What changes between Squad and Fleet is **governance**, not technology: a Fleet needs explicit Store curation, signed agent versions, and a central review board for shared agents — a Squad does not.

Fleet ≠ N Squads in parallel. A Fleet shares state across Squads: when the Auth agent learns a new tenant-isolation pattern in Squad A, every Squad B running tomorrow picks it up. That cross-Squad compounding is what makes Fleet a qualitatively different pattern — not just "more developers".

5.4 3.4 Spec-Driven Development — making the spec the source of truth

The Squad pattern only works if the agents have a single, unambiguous reference. That reference is the spec. **Spec-Driven Development (SDD)** is the discipline that turns this from a convention into an engineering process — the same way DevOps turned ad-hoc scripts into pipelines.

For an enterprise system (React frontends + backend services + data layer + shared libs + AI agents), Microsoft now recommends layered, modular specs rather than a single monolithic document. The fit with multi-agent development is exact: agents consume structured context best, and machine-readable specs *are* that context.¹

5.4.1 Three spec layers

1. **Business & domain layer** — technology-agnostic. Captures business capabilities, domain language (DDD bounded contexts), business rules, non-functional requirements (perf, security, compliance, SLAs).

¹Adapted from Lee Stott, *Spec-Driven Development for AI-Enabled Enterprise Systems*, Microsoft Tech Community, May 2026 — <https://techcommunity.microsoft.com/blog/educatordeveloperblog/spec-driven-development-for-ai-enabled-enterprise-systems/4520807>.

2. **Solution & architecture layer** — system context (C4 diagrams), service boundaries and ownership, integration patterns, event flows, data ownership and high-level models.
3. **Implementation-oriented specs per component** — one focused spec per:
 - **Frontend / UI (React)**: screen catalogue, UX flows, state contracts, API dependencies, validation rules, accessibility, performance budgets.
 - **API / service**: OpenAPI or AsyncAPI contracts, error models, auth, rate limits, SLAs, observability requirements.
 - **Database / schema**: logical data model, ownership per service, migration strategy, retention, indexing, partitioning.
 - **Shared libraries**: responsibilities, versioning policy, compatibility matrix.
 - **Integrations**: protocols, payloads, sequencing, idempotency, retry/backoff, failure modes.

In practice: **one master business + architecture spec per domain, separate specs per service / module**, everything linked via stable IDs (REQ-123, SVC-ORDER-001) so you can trace a requirement to its spec, its implementation, and its tests.

5.4.2 Base template (every spec)

Purpose & scope · Stakeholders & dependencies · Requirements mapping (IDs covered) · Architecture & interaction overview · Contracts (APIs, events, data) · Non-functional requirements · Risks & open questions · Test & acceptance criteria.

Each component type extends this base with its own sections (UX flows for frontend, OpenAPI link for API, schema + migration plan for DB, sequence diagrams + idempotency rules for integrations).

5.4.3 Contracts are first-class citizens

- API contracts via **OpenAPI / AsyncAPI** — the spec is the source of truth, not the code.
- **Contract tests** keep providers and consumers aligned.
- Versioning is **explicit** (v1, v2) — never breaking changes in place.
- Database migrations are **code** (EF Core, Flyway, Liquibase) and documented backward-compatibility windows let APIs serve old + new fields for a defined period.
- Prefer sharing **contracts** (OpenAPI, JSON Schema) over large shared code libraries; when code must be shared, version the library independently and document compatibility (**Service A supports SharedLib 2.x**).

5.4.4 The SDD workflow

Discovery → Business & Architecture specs → Contract design (OpenAPI / events / schemas)
 → Task generation (AI agent reads spec → emits work items)
 → Implementation (code satisfies spec; spec stays the reference)
 → Validation (contract tests, unit, integration, E2E – all trace back to spec IDs)
 → Architecture review against the spec; update spec if reality diverges
 → Release + observability (dashboards / alerts tied to specified SLIs and SLOs)

5.4.5 Suggested repository layout

/docs

```

/business
/architecture
/decisions          ← Architecture Decision Records (ADRs)
/specs
/frontend
/services
  /orders
  /customers
/integrations
/data
/src
  /frontend
  /services
  /shared
/tests
/ops
  /pipelines
  /infra-as-code

```

5.4.6 Governance — avoiding drift

- **Traceability** — IDs everywhere: requirement → spec section → task → PR → test → release.
- **ADRs** — Architecture Decision Records for every non-trivial design choice.
- **Spec checks in CI** — API implementation must match OpenAPI, DB schema must match migration plan, generated clients must be up to date; CI **fails** when they diverge.
- **Definition of Done** — spec updated · tests linked · contracts validated.
- **Spec health reviews** — recurring sweep for drift and stale sections.

5.4.7 Agents + SDD = high-fidelity context

This is the punchline for our Squad model:

- **MCP servers expose the spec repo** as tools (`get_openapi(service, version)`, `get_adrs(domain)`, `get_requirements(id)`). Agents query the **true** source of truth instead of hallucinating.
- The Squad maps cleanly onto the SDD pipeline:
 - **Spec agent** reads and validates specs · **Implementation agent** generates code against the OpenAPI · **Test agent** generates contract + unit tests from acceptance criteria · **Review agents** check spec ↔ code conformance.
- **Constrain agents to spec-linked files only.** An agent working on `SVC-ORDER-001` should not edit files outside that spec's scope.
- Add **automated checks** that compare generated code to contracts and fail the build when they diverge.

5.4.8 Where to start — don't boil the ocean

Pick **one domain** (e.g. `Orders`), design a thin end-to-end SDD flow — spec → contract → tasks → code → tests — run it with agents in the loop, learn, iterate. Roll out the patterns once the first slice feels natural. The Spec Kit (§ 3.2) is the right tool to bootstrap this first slice.

5.5 3.5 Agents vs Skills

- **Agents** are autonomous, model-driven, with tools and memory. They reason, plan, ask questions.
- **Skills** are declarative scripts the agent calls. They are deterministic and cheap.

The right pattern is Agent + Skill: the agent reasons about *what* and *why*, the skill executes the *how* reproducibly.

5.6 3.6 Coding with agents — best practices

Three patterns that compressed the 7.5-hour integration phase described in § 2.3 — and that we now apply by default.

1. Start every feature with a `plan.md`.

Before issuing code-generation calls, the conductor (or the Lead agent) writes a `plan.md` capturing:

- The user-visible outcome (“the user clicks Insert, a PNG lands in the slide”)
- The contracts at each seam (REST routes, JWT claims, environment variables, Bicep outputs)
- An **explicit task split** — one task per agent — with the **parallelisable subset clearly marked**
- Acceptance tests per task

The `plan.md` is the single source of truth. Agents read from it. Re-runs read from it. Cost is dominated by re-prompting; a good `plan.md` eliminates ~80 % of re-prompts.

2. Explicitly mark parallel vs sequential work.

```
## Plan – feature: workspace browser
- [P] Frontend: workspace tree component           (Frontend agent)
- [P] Backend: /api/workspaces route + service    (Backend agent)
- [P] Infra:   Bicep module for Static Web App    (Infra agent)
- [S] Auth:    OBO token wiring through middleware (Auth agent – depends on
  ↪ Backend)
- [S] Tests:   Jest + Playwright                  (Tester agent – depends
  ↪ on Frontend + Backend)
```

[P] = parallel, [S] = sequential. The CLI dispatches [P] tasks concurrently and serialises [S] ones. Without this annotation, the Lead agent has to infer the dependency graph from prose — error-prone and token-expensive.

3. Kernel first, then reason by feature (multi-developer pattern).

The most common failure mode is to ask eight agents to build eight verticals on day one. They produce eight locally-correct fragments with eight different contracts. The pattern that works:

- **Day 1 — Kernel.** Frontend shell, one backend route, auth middleware, deploy pipeline, smoke test. **One vertical end-to-end.** All agents converge on the same contracts.
- **Day 2+ — Features.** Once the kernel exists, every new feature is a focused Squad sprint: one `plan.md`, one parallel fan-out, one merge. Contracts are inherited from the kernel.

This is the same walking-skeleton pattern that works for multi-human teams. It scales to multi-agent teams **without modification** — because the constraint is identical: shared contracts must exist *before* parallel work pays off.

5.7 3.7 Testing strategy

Tests are not an afterthought — they are how the Squad knows it is done.

Layer	Tool	Generated by	Run by
Unit	Jest	Tester agent	CI on every PR
Integration	Jest + supertest (API routes)	Tester agent	CI on every PR
E2E (UI)	Playwright	Tester agent	CI on <code>main</code> push
Contract (OBO scopes, Bicep outputs)	Skills (deterministic scripts)	Infra / Auth agents	CI + pre-deploy
Smoke (post-deploy)	Invoke- RestMethod /api/health	Conductor	After <code>azd up</code>

In this project, the Tester agent produced **68 tests** during the scaffold phase. They drove the bug discovery in § 2.3: every fix landed with a failing test first, a passing test second.

Three rules for agent-generated tests:

1. **Tests must come from a different agent than the code.** The Tester is a separate persona with its own context — it reads the spec, not the implementation. If the same agent writes both, it tests what it wrote, not what was specified.
2. **Coverage is a leading indicator, not a goal.** The Tester reports coverage; the Conductor uses it to spot under-tested modules but does not chase 100 %.
3. **The AI code review pass owns test review.** One of the four review agents explicitly asks: *are the tests testing the spec, or are they testing the implementation?*

6 4. The Cost Model

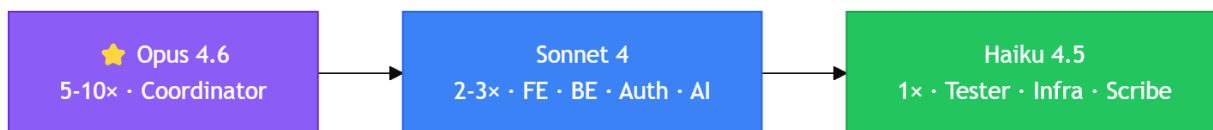


Figure 8: Cost tiers

6.1 4.1 AI Credits

Since **June 1, 2026**, the Copilot subscription bills on **GitHub AI Credits** (1 credit = \$0.01 USD), replacing the previous “premium request” model. Consumption is metered in **tokens** (input, output, cached) at per-model published rates, with monthly allotments by plan: Pro 1 000 credits, Pro+ 3 900, Business 1 900/seat, Enterprise 3 900/seat. Code completions and Next Edit suggestions remain **unlimited** and do not consume credits.

Heavy-reasoning calls (Opus 4.7, GPT-5.4) burn credits 5–10× faster than structural work on Haiku 4.5 or GPT-5.4-mini. Cached input tokens are billed up to 10× cheaper than fresh input. Optimising the model mix is half the cost story; the other half is **caching, RAG, and prompt economy** — which become first-class IP assets under token-based billing, on par with the agents themselves.

6.2 4.2 Cost comparison

	Traditional SI	System Agency
Effort	43 man-days	13.3 h human + agent compute
Loaded cost	~ \$31,000 (3 senior devs × 5 days × \$720 + 2 specialists × \$720 × 4 + PM 2 days)	~ \$2,340 (\$1,800 compute ≈ 180 000 AI credits — ~60 % Sonnet/Opus reasoning, ~30 % Haiku structural, ~10 % cached input — plus \$540 human supervision)
Margin	~ 30 % on T&M	~ 92 % on outcome billing
Lead time	6-8 weeks calendar	2 days

Numbers are illustrative — they vary by project and by how much of the Agent Store is reusable. The structure does not.

7 5. From SI to System Agency

7.1 5.1 The shift

The classic System Integrator sells **man-days**. The economics work when delivery cost ≈ delivery price minus ~30 % margin. Generative agents drop the delivery cost by an order of magnitude. If the price stays anchored to man-days, margins collapse to zero. If the price anchors to **outcomes**, margins explode.

That is the System Agency model: **price the outcome, not the input**.

7.2 5.2 Why now

Four converging forces:

1. **Models** — Sonnet 4 / Opus 4.6 / Haiku 4.5 cross the threshold for full feature delivery, not just code completion.
2. **Tooling** — Copilot CLI, Spec Kit, MCP, Agent Store make Squad orchestration accessible to a single developer.
3. **Buyer expectation** — clients have started benchmarking AI-augmented vendors against traditional ones. The price gap is becoming visible.
4. **Talent** — senior engineers gravitate towards agency models where they capture upside.

7.3 5.3 Old world vs new world

	Old (SI)	New (System Agency)
Unit	Man-day	Outcome / feature
Pricing	Time & materials	Fixed-price per outcome
Margin	25-35 %	70-90 %
Team	5-50 humans	1 human + Squad of agents
Differentiator	Headcount	Agent Store + IP
Sales cycle	RFP, proposals	Show, don't tell

7.4 5.4 Three phases

1. **Copilot phase** — agents assist humans. Productivity 2-3×. Most teams are here.
2. **Agency phase** — agents deliver outcomes; humans supervise. Productivity 10-30×. This project sits here.
3. **Autonomous phase** — agents close the full loop with the customer; humans set strategy. Productivity 100×+. Emerging.

7.5 5.5 Encoding IP

The competitive moat shifts from *people* to **agent definitions, prompts, knowledge bases, and Skills**. Every project enriches the Agent Store. Every Spec Kit becomes a template. Every fix in code review becomes a memory. Compounding is the new differentiator.

7.6 5.6 Emerging roles

- **Spec Author** — translates business intent into Spec Kit documents.
- **Agent Engineer** — designs, forges, and maintains agents.
- **Conductor / Squad Lead** — orchestrates agents through the CLI.
- **Reviewer** — domain expert who validates outcomes; not a line-by-line code reviewer.
- **Rainmaker** — sells outcomes, not headcount.

7.7 5.7 Getting started

1. Pick one **non-critical** project. Run it end-to-end with a Squad.
2. Capture **everything** — prompts, agent definitions, memories, fixes — in the Agent Store.
3. Re-use on the second project. Measure compression.
4. After three projects, refactor the Store. Promote the most-reused agents to first-class.
5. Start pricing the **fourth** project on outcome.

7.8 5.8 Lessons learned

- **Scaffold is free, integration is the work.** Plan calendar around the seams between agent outputs.
- **Pick the right model per role.** Haiku for structure, Sonnet/Opus for reasoning.
- **AI code review is non-negotiable.** Four parallel agents catch what one human misses.
- **The CLI is the conductor.** Don't let the agents run unsupervised on critical paths.

- **Spec quality dominates.** A bad spec produces eight bad agents in parallel.
- **One plan.md per feature.** Mark every task [P] or [S]. The CLI parallelises on the annotation, not on guesses.
- **Build the kernel first.** One end-to-end vertical before fanning out — same walking-skeleton pattern as multi-human teams.
- **Different agents for code and tests.** The Tester reads the spec, not the diff.

8 6. Conclusion

A working PowerPoint add-in, fully integrated with Microsoft Fabric, GPT-4o Vision, Entra ID, Bicep, and CI/CD — built in **13.3 hours** by **one developer** orchestrating **eight agents**. The artefact matters less than the production system that made it possible: Spec Kit, Agent Forge, Squad, Copilot CLI, Agent Store, Fleet.

The same playbook scales. The compression is real. The margin shift is structural. The System Integrator model is not dead — but the System Agency model is what wins the next decade.

9 Resources

- Source repository — <https://github.com/fredgis/OfficeAddin>
- GitHub Copilot CLI — <https://github.com/github/copilot-cli>
- Spec Kit — <https://github.com/github/spec-kit>
- Agent Forge — <https://github.com/microsoft/agent-forge>
- Squad — <https://github.com/bradygaster/squad>
- GitHub Copilot AI Credits billing — <https://docs.github.com/en/copilot/concepts/billing>
- Spec-Driven Development for AI-Enabled Enterprise Systems (Microsoft Tech Community, May 2026) — <https://techcommunity.microsoft.com/blog/educatordeveloperblog/spec-driven-development-for-ai-enabled-enterprise-systems/4520807>
- Azure microservices & DDD — <https://learn.microsoft.com/azure/architecture/microservices/>
- Azure API design best practices — <https://learn.microsoft.com/azure/architecture/best-practices/api-design>
- Model Context Protocol (MCP) for beginners — <https://aka.ms/mcp-for-beginners>
- Microsoft Fabric — <https://learn.microsoft.com/fabric/>
- Power BI Export API — <https://learn.microsoft.com/rest/api/power-bi/reports/export-to-file>
- Office Add-ins (manifest, taskpane, SSO) — <https://learn.microsoft.com/office/dev/add-ins/>
- Azure OpenAI GPT-4o Vision — <https://learn.microsoft.com/azure/ai-services/openai/concepts/gpt-with-vision>
- Static Web App auth — <https://learn.microsoft.com/azure/static-web-apps/authentication-authorization>